

Agentic RAG System — Technical Write-Up

1. Architecture Decisions

1.1 Modular Pipeline Design

The system is designed using a modular pipeline architecture consisting of separate components for document ingestion, chunking, embedding generation, vector storage, retrieval, and response generation. Each module works independently, making the system easier to maintain, debug, and scale. This approach also allows components such as the vector database or embedding model to be replaced without affecting the overall workflow.

1.2 Document Processing and Chunking

The chatbot supports multiple document formats including PDF, TXT, and CSV files. During ingestion, documents are converted into plain text and divided into smaller chunks before embedding generation.

A recursive character-based chunking strategy was used because it balances retrieval quality and implementation simplicity. Chunk overlap is maintained to preserve contextual continuity between chunks and improve retrieval accuracy.

1.3 Embedding Model Selection

The project uses the sentence-transformers model `'all-MiniLM-L6-v2'` for embedding generation. This model was selected because it is lightweight, fast, and performs well on semantic similarity tasks. Since the model runs locally, it avoids additional API costs while still providing reliable vector representations.

1.4 Vector Database

ChromaDB was chosen as the vector database for storing embeddings and performing similarity search. It integrates easily with Python applications, supports persistent storage, and provides efficient semantic retrieval capabilities for small- to medium-scale datasets.

Cosine similarity is used to compare query embeddings with document embeddings to identify the most relevant chunks.

1.5 Retrieval-Augmented Generation (RAG)

The system follows a Retrieval-Augmented Generation workflow. When a user submits a query, the retriever searches the vector database and returns the most semantically relevant chunks. These chunks are then passed to the LLM as context for generating the final answer.

This design helps the chatbot generate grounded responses based on document content rather than relying solely on the LLM's internal knowledge.

1.6 Hallucination Prevention

A major requirement of the assignment was minimizing hallucinations. To address this, the system includes multiple safeguards:

- The LLM is instructed to answer only from retrieved context.
- Similarity threshold filtering removes low-confidence retrievals.
- If no relevant chunks are found, the system returns a fallback response instead of generating unsupported answers.

Example fallback response:

"I don't have enough information in the provided documents."

1.7 Multi-Format Support

The ingestion pipeline supports:

- PDF documents
- TXT files
- CSV datasets

This allows the chatbot to handle different types of enterprise or research data within a single system.

2. Limitations

2.1 Large Document Processing Time

Processing large research papers can take significant time because embeddings must be generated for every chunk. Performance decreases further when many documents are uploaded simultaneously.

2.2 Limited Understanding of Images and Tables

The current PDF extraction process only handles textual information. Images, diagrams, and complex tables inside PDFs are not interpreted correctly, which may reduce answer accuracy for visually dependent documents.

2.3 Chunk Boundary Issues

Although chunk overlap is used, important information can still be split across multiple chunks. In such cases, retrieval quality may decrease if the relevant context is divided improperly.

2.4 Dependency on Embedding Quality

The quality of retrieval depends heavily on the embedding model. If embeddings do not accurately represent document meaning, retrieval precision can decrease.

2.5 Basic User Interface

The current implementation uses a command-line interface and basic Colab execution flow. It is functional but not optimized for production-grade user experience.

2.6 API Dependency

The system depends on OpenAI API access for response generation. Internet connectivity and API availability are required during execution.

3. Suggestions for Scaling

3.1 Improved Retrieval Techniques

Retrieval performance can be improved using:

- Hybrid search combining keyword and semantic search
- Cross-encoder re-ranking
- Query expansion techniques
- Semantic chunking

These methods can significantly improve answer relevance and retrieval precision.

3.2 Production Deployment

For large-scale deployment, the system can be migrated to:

- FastAPI backend
- React frontend
- Cloud-hosted vector databases such as Pinecone, Weaviate, or Qdrant

This would support better scalability and concurrent users.

3.3 Conversation Memory

Future versions can maintain conversation history to support follow-up questions and more natural interactions.

3.4 Automated Ingestion Pipeline

A file watcher or scheduled ingestion system can automatically detect and process newly added documents without manual intervention.

3.5 Authentication and Security

The current system has no authentication layer. Production deployments should include:

- User authentication
- Access control
- Secure API handling
- Encrypted storage

3.6 GPU Acceleration

Embedding generation and retrieval can be accelerated using GPU-based infrastructure, especially for large document collections.

4. Conclusion

This project demonstrates a complete end-to-end Agentic RAG pipeline capable of ingesting multiple document formats, generating embeddings, storing vectors, retrieving contextual information, and producing grounded responses using an LLM.

The system emphasizes modularity, retrieval accuracy, and hallucination prevention while maintaining a practical and scalable architecture. The implementation successfully satisfies the assignment requirements and provides a strong foundation for future improvements such as web deployment, advanced retrieval methods, and conversational memory.